

NAME

OpenFEGenerateLigandNetwork.py - Generate ligand network

SYNOPSIS

```
OpenFEGenerateLigandNetwork.py [--infileParams <Name,Value,...>] [--loggingLevel <Info or Warning>] [
--mapper <mapper1, mapper2,...>] [--mapperParams <Name,Value,...>] [--mapperScorer <LOMAP>] [
--network <text>] [--networkParams <Name,Value,...>] [--outfilePrefix <text>] [--overwrite] [-w <dir>] -i
<infile> -o <outfiledir>
```

OpenFEGenerateLigandNetwork.py -h | --help | -e | --examples

DESCRIPTION

Generate a ligand network for molecules in an input file and write it out as graphml and image files under the output directory. You may optionally generate image files for all edges in a ligand network.

You must specify a valid input file containing 3D coordinates for all molecules. In addition, the hydrogens must be present for all molecules in the input file.

The supported input file format is: SD (.sdf, .sd)

The supported output file formats are: GraphML (.graphml), SVG (.svg), PNG (.png), etc.

Possible output directories:

```
<OutfileDir>
<OutfileDir>/Edges
```

Possible outfile prefix:

```
<OutfilePrefix> or <InfileRoot>
```

Possible output files:

```
<OutfilePrefix>_Network_<NetworkName>_Mapper_<MapperNames>.graphml
<OutfilePrefix>_Network_<NetworkName>_Mapper_<MapperNames>.<ImgExt>
```

Edges:

```
<MolName1>_To_<MolName2>_<NetworkName>_Mapper_<MapperNames>.png
... ..
```

OPTIONS

-e, --examples

Print examples.

-h, --help

Print this help message.

-i, --infile <infile>

Input file name.

--infileParams <Name,Value,...> [default: auto]

A comma delimited list of parameter name and value pairs for reading molecules from files. The supported parameter names for different file formats, along with their default values, are shown below:

SD: removeHydrogens,no,sanitize,yes,strictParsing,yes

--loggingLevel <Info or Warning> [default: Warning]

Logging level to configure the 'root logger' via logging.basicConfig() function. The default logging level is changed from 'logging.INFO' to 'logging.WARNING'. Otherwise, OpenFE and its associated modules may

generate a lot of informational messages.

-m, --mapper <mapper1, mapper2> [default: LOMAP]

A comma delimited names of atom mappers to use for generating a ligand network. Possible values: LOMAP [Lead Optimization MAPer; Ref 176] or Kartograf [Ref 177]. You may specify multiple mappers for generating mapping between two molecules. All specified mappers are employed to identify the highest scoring edges for generating a ligand network.

--mapperParams <Name,Value,...> [default: auto]

A comma delimited list of parameter name and value pairs for atom mappers used during the generation of a ligand network.

The supported parameter names along with their default values are shown below:

```
lomapTime, 20, [ Units: seconds ]
lomapThreeD, yes [ Possible values: yes or no ]
lomapMax3D, 1.0 [ Units: Angstrom ]
lomapElementChange, yes [ Possible values: yes or no ]
lomapSeed, None [ Possible value: A string. An empty string causes
    MCS search to start from scratch ]
lomapShift, no [ Possible values: yes or no ]

kartografAtomMaxDistance, 0.95 [ Units: Angstrom ]
kartografAtomMapHydrogens, yes [ Possible values: yes or no ]
kartografMapHydrogensOnHydrogensOnly, No [ Possible values: yes or
    no ]
kartografMapExactRingMatchesOnly, yes [ Possible values: yes or no ]
kartografAllowPartialFusedRings, yes [ Possible values: yes or no ]
```

A brief description of parameters is provided below:

```
lomapTime: Time out for MCS algorithm.
lomapThreeD: Use atom positions to prune symmetric mappings.
lomapMax3D: Forbid mapping between atoms with distance more than
    specified value.
lomapElementChange: Allow mappings that change an atom element.
lomapSeed: An Empty SMARTS string causes MCS search to start from
    scratch.
lomapShift: Keep pre-aligned atom positions for 3D position checks.

kartografAtomMaxDistance: Geometric criteria for two atoms
    corresponding to maximum distance between them.
kartografAtomMapHydrogens: Map hydrogens.
kartografMapHydrogensOnHydrogensOnly: Map hydrogens only on
    hydrogens.
kartografMapExactRingMatchesOnly: Map rings with only matching ring
    size and bond orders. In addition, ring breaking is not
    permitted.
kartografAllowPartialFusedRings: Allow mapping of partially fused
    rings.
```

--mapperScorer <LOMAP> [default: LOMAP]

Atom mapper scorer to use for scoring edge mappings during generation of a ligand network. Possible value: LOMAP. The atom scorer is not used during the generation of MinimalSpanning network.

-n, --network <text> [default: MinimalSpanning]

Name of a ligand network to generate. Possible values: LOMAP, MinimalSpanning or Radial.

--networkParams <Name,Value,...> [default: auto]

A comma delimited list of parameter name and value pairs for generating a ligand network.

The supported parameter names along with their default values are shown below:

```
lomapDistanceCutoff, 0.4
lomapMaxPathLength, 6
```

```

lomapRequireCycleCovering, yes [ Possible values: yes or no ]

minimalSpanningProgress, no [ Possible values: yes or no ]

radialCentralligand, None [ Possible values: Valid ligand name ]

outputEdges, no [ Possible values: yes or no ]
outputNetworkFormat, svg [ Possible values: Any valid format. ]

```

A brief description of parameters is provided below:

```

lomapDistanceCutoff: Maximum distance/dissimilarity between two
                     molecules for an edge to be accepted.
lomapMaxPathLength: Maximum distance between any two molecules in
                     the resulting network
lomapRequireCycleCovering: Add cycles into the network

minimalSpanningProgress: Show progress using tqdm.

radialCentralligand: Name of central ligand. A valid ligand name
                     must be specified to generate a radial ligand network.

outputEdges: Generate PNG image files for all edges in a ligand
              network.
outputNetworkFormat: Valid image file format for ligand network.
                     You must specify a valid format supported by Python module
                     Matplotlib. For example: PNG (.png), SVG (.svg), PDF (.pdf),
                     etc. In addition, the graphml file is always generated.

```

```
-o, --outfileDir <outfiledir>
```

Output directory.

```
--outfilePrefix <text> [default: auto]
```

Prefix for generating output files under output directory.

```
--overwrite
```

Overwrite existing files.

```
-w, --workingdir <dir>
```

Location of working directory which defaults to the current directory.

EXAMPLES

To generate a minimal spanning ligand network for molecules in a SD file with 3D structures, employing LOMAP atom mapper to map edges, and write out network GraphML, SVG and edge image files to output directory, type:

```
% OpenFEGenerateLigandNetwork.py -i SampleTyk2Ligands.sdf
-o SampleTyk2LigandsMSTNetwork
```

To run the first example along with generating PNG image files for all edges, type:

```
% OpenFEGenerateLigandNetwork.py -i SampleTyk2Ligands.sdf
-o SampleTyk2LigandsMSTNetwork --networkParams "outputEdges, yes"
```

To run the first example employing Kartograf atom mapper to map edges, and write out various output files under to directory, type:

```
% OpenFEGenerateLigandNetwork.py -i SampleTyk2Ligands.sdf
-o SampleTyk2LigandsMSTNetwork -m Kartograf
```

To run the first example for generating a radial ligand network centered around the ligand lig_ejm_31, and write out various output files to output directory, type:

```
% OpenFEGenerateLigandNetwork.py -i SampleTyk2Ligands.sdf
-o SampleTyk2LigandsRadialNetwork -n Radial --networkParams
"RadialCentralLigand, lig_ejm_31"
```

To run the previous example along with generating PNG image files for all edges, type:

```
% OpenFEGenerateLigandNetwork.py -i SampleTyk2Ligands.sdf
-o SampleTyk2LigandsRadialNetwork -n Radial --networkParams
"RadialCentralLigand, lig_ejm_31, outputEdges, yes"
```

To run the first example by specifying explicit values for various parameters and write out various output files to output directory, type:

```
% OpenFEGenerateLigandNetwork.py -i SampleTyk2Ligands.sdf
-o SampleTyk2LigandsLOMAPNetwork
--loggingLevel Warning -m LOMAP --mapperParams "lomapTime, 20,
lomapThreeD, yes, lomapElementChange, yes, lomapElementChange, yes,
lomapSeed, None, lomapShift, no" -n LOMAP --networkParams
"lomapDistanceCutoff, 0.4, lomapMaxPathLength, 6,
lomapRequireCycleCovering, yes, outputEdges, yes"
```

AUTHOR

Manish Sud(msud@san.rr.com)

SEE ALSO

OpenFECalculateAbsoluteHydrationFreeEnergy.py, OpenFECalculateRelativeBindingFreeEnergy.py,
OpenFECalculateRelativeHydrationFreeEnergy.py, OpenFECalculatePartialCharges.py

COPYRIGHT

Copyright (C) 2026 Manish Sud. All rights reserved.

The functionality available in this script is implemented using OpenFE, an open source molecular for alchemical free energy calculations.

This file is part of MayaChemTools.

MayaChemTools is free software; you can redistribute it and/or modify it under the terms of the GNU Lesser General Public License as published by the Free Software Foundation; either version 3 of the License, or (at your option) any later version.